

KAAMSETU Implementation Blueprint v1.0

KaamSetu should launch as a **managed, trust-first, hyperlocal operating system**, not as an open marketplace and not as a full-stack Urban Company clone. In practical terms, that means: one city first, a very small service catalog, invite-only worker onboarding, controlled customer access, heavy admin control, assisted booking through WhatsApp and phone, and a thin app layer on top of strong operational workflows. The reason is not ideology; it is execution reality. India's GST treatment of e-commerce operators becomes materially more complex when the platform collects consideration for third-party supplies, DPDP compliance requires disciplined notice, consent, security, grievance handling, and erasure workflows, and affordable WhatsApp, IVR, OTP, and payment-link infrastructure already exists so KaamSetu does not need to build everything as a fully automated consumer app on day one. ¹

The strongest execution stance is therefore this: **KaamSetu v1 is a software-enabled operations business**. Software should reduce chaos, create records, and make dispatch, verification, support, and payments auditable. It should not try to replace human judgment in the first 90 days. That is the core design choice that keeps the company cheap, controllable, and survivable.

Strategic execution verdict

What KaamSetu v1 actually is

KaamSetu v1 should be a **closed-beta Android-first managed marketplace** with four booking modes: **fixed-price task, quote-based job, shift-based labour booking**, and **assisted booking through WhatsApp/phone**. The booking rails should exist across mobile, WhatsApp, and IVR because Meta's WhatsApp Business Platform supports interactive messages and Flows, and entry-level IVR/contact-center products in India are affordable enough for a bootstrap operator to support call-assisted demand from the beginning. ²

The product should serve three demand patterns from day one:

Demand pattern	Examples	Pricing mode	Dispatch mode
Standard household fix	leaking tap, fan install, switch repair, basic carpentry fix	fixed-price	semi-automatic
Variable-scope job	painting, masonry, waterproofing, fabrication, major appliance diagnosis	quote-based	ops-assisted
Labour/shift need	helper, loader, mover, site helper, cleaning support	shift-based	manual plus batch dispatch

This hybrid model is the right answer to the founder's earlier question about fixed prices versus open marketplace. **Do not run a free-for-all bidding bazaar.** Use fixed prices where scope is repeatable, quotes where scope is variable, and shift cards where labour hours are the real product.

What KaamSetu v1 is not

KaamSetu v1 should **not** be:

Not in v1	Why
Nationwide launch	kills density and support quality
iOS app	doubles QA and release complexity for little early benefit
real-time map-heavy consumer UX	avoid cost and complexity until dispatch is stable
open bidding between many workers	creates price chaos, low trust, and support pain
in-app wallet or stored balance	unnecessary regulatory and reconciliation burden
instant automated refunds everywhere	too risky before dispute ops harden
public self-serve worker onboarding	raises fraud/quality risk
20+ service categories	destroys training, pricing control, and fill rate
ads marketplace	damages trust before liquidity exists

This also means **no investor-style "super app" scope**. The product should begin with a narrow promise and fulfill it reliably.

Feature prioritization matrix

Must have	Should have	Nice to have	Postpone
customer booking, address capture, job types, worker profile, dispatch queue, worker availability, OTP start/end, payment recording, ratings, complaints, admin notes, audit trail, invite codes, language toggle	WhatsApp booking automation, IVR routing, quote comparison, before/after photos, B2B account tagging, referral codes, worker earnings ledger	masked calling, route ETA, coupon engine, worker learning module, smart rebooking, category-specific checklists	wallet, escrow ledger, subscription engine automation, AI pricing, dynamic surge pricing, iOS app, full CRM, advanced analytics suite

The build order should follow the operational loop: **worker verification → booking intake → dispatch → OTP-protected job execution → payment record → rating/dispute → settlement/revenue reconciliation.**

The non-negotiable monetization stance for execution

For the first implementation cycle, KaamSetu should **not** depend on ads and should **not** start with a worker-punishing lead-fee model. It should also avoid a platform-held wallet. The early monetization stack should be:

- a small **customer booking fee / support fee** on completed or confirmed bookings,
- **worker membership only after workers see real value**,
- **B2B recurring accounts** for shops, warehouses, clinics, schools, builders, and small facilities,
- and only later, **low commission on platform-collected fixed-price jobs**.

This is the most trust-preserving path while still moving toward self-sustaining revenue.

Product and operations blueprint

Customer app blueprint

The customer app should be intentionally thin. The app is not where KaamSetu wins at launch; the ops layer is.

The v1 customer flow should have only seven core screens:

Screen	Purpose
home	choose service type: fixed, quote, shift
address	locality, landmark, pincode, optional map pin
booking details	issue, photos, preferred slot, language
confirmation	expected arrival window, service mode, worker profile
live job	assigned worker, support call, OTP start/end
payment	cash / UPI / payment link / token status
post-job	rating, reorder, complaint

For Bharat-first usability, the customer should be able to finish a booking in under two minutes with **icons + short text + voice-note option**. DPDP also makes multilingual notices important: the law explicitly requires the notice accompanying consent to be accessible in English or any Eighth Schedule language, which supports the decision to launch in English + one city language from day one. ³

The app should also expose a clear **“Book on WhatsApp”** fallback for every user. Once Cloud API is live, use **interactive list messages** and eventually **WhatsApp Flows** for category selection, address capture, and slot confirmation; Meta’s documentation specifically supports interactive messages and Flows for task-centric workflows. In phase 0, however, use the normal WhatsApp Business app manually and move to Cloud API only after the operational workflow is proven. ⁴

Worker app blueprint

The worker app matters more than the customer app in v1, because KaamSetu's reliability depends on worker response, punctuality, proof, and reporting.

The worker app should have these core modules:

Module	What it does
availability	online/offline, service area, shift window
jobs today	accepted, pending, completed, cancelled
job detail	address, issue summary, customer language, photos
action buttons	accept, arrive, start, pause, complete, issue raised
proof capture	before/after photos, material used, quote photos
KYC/status	document status, verification tier, training flags
earnings	booking count, pending dues, incentives, membership status
support	call admin, dispute flag, emergency help

The UI should have **large touch targets, category icons, low text density, and one-tap state changes**. This is also why Android should be the first and only worker platform in the first phase. Flutter supports app internationalization through its standard localization workflow, which is enough for English plus one or two Indian languages without creating separate codebases. ⁵

Admin panel blueprint

The admin panel is the real control tower. Build this first.

Django's official docs explicitly describe the admin interface as one of its strongest features and recommend it as an internal management tool. That is exactly KaamSetu's need in a bootstrap launch. ⁶

The admin panel should include:

Area	Minimum capability
worker ops	KYC, skills, service areas, status, quality flags
jobs board	new, dispatched, accepted, live, completed, disputed
dispatch	worker list by zone/category, manual override, batch send
pricing	fixed-price catalog, inspection fee, shift cards, project templates
complaints	issue type, evidence, notes, SLA timer, resolution

Area	Minimum capability
payments	booking fees, payment links, cash entries, dues, invoices
growth	invite codes, referrals, channel tags, B2B account tags
audit	job events, admin actions, OTP logs, communication logs

Without this panel, the rest of the product stack will drift.

Booking flow

The v1 booking flow should be:

1. Customer chooses **fixed-price, quote-based, or shift-based**.
2. Customer enters address, landmark, and optionally uploads photos.
3. System creates a **job record** with category, zone, urgency, and booking channel.
4. Dispatch engine selects eligible workers in that microzone.
5. Workers receive the job in priority order or in a small batch.
6. First confirmed worker is reserved.
7. Customer sees worker profile and time window.
8. Worker arrives and starts only after **customer OTP**.
9. Job completes with photos and final amount rules based on booking type.
10. Payment is recorded.
11. Rating and complaint window opens.

The OTP step is important. It reduces fake arrival claims, fake completions, and many “worker never showed up” disputes.

Dispatch flow

Do **not** attempt full algorithmic dispatch on day one. Build **algorithm-assisted, human-overridable dispatch**.

Recommended v1 dispatch logic:

- Filter by **zone + category + verification tier + availability**.
- Rank by **recent jobs served, rating/complaint score, response speed, and repeat-customer block**.
- For fast fixed-price jobs, notify **top 3 workers** for 60–90 seconds.
- If nobody accepts, send to the next 5 and alert ops.
- For quote jobs, assign **up to 3 site-visit candidates**, then let the customer choose from submitted quotes.
- For shift jobs, dispatch in **small batches** and require customer confirmation before finalizing the roster.
- Admin always has **manual reassignment power**.

PostgreSQL plus PostGIS is enough for this. PostGIS extends PostgreSQL to store, index, and query geospatial data, which is exactly what KaamSetu needs for microzone dispatch. PostgreSQL also natively supports JSONB and full-text search, which are useful for flexible job metadata and admin search. ⁷

Worker verification flow

Launch with **tiered verification**, not fake “100% verified” marketing.

Recommended verification tiers:

Tier	Checks	Can do
Bronze	phone, photo, bank proof, one govt ID, one local reference	low-risk exterior or supervised tasks
Silver	Bronze + second reference + skill test/demo + address validation	normal household jobs
Gold	Silver + police check or prior employer/contractor certificate + higher complaint threshold	sensitive in-home work, premium categories

UIDAI’s Aadhaar Paperless Offline e-KYC is far safer than collecting and storing raw Aadhaar copies. UIDAI states that offline e-KYC is voluntary, does not reveal the full Aadhaar number, and lets service providers verify identity without core biometrics; it also warns service providers not to share or display the XML, share code, or its contents. KaamSetu should therefore prefer **offline Aadhaar XML/QR or alternative ID documents**, and should avoid storing full Aadhaar numbers in the MVP database. ⁸

KaamSetu should also encourage workers to complete **e-Shram** self-registration rather than trying to become the welfare system itself. The Labour Ministry and e-Shram materials show that platform-worker and aggregator workflows are evolving, and the e-Shram ecosystem already includes platform-worker registration and aggregator modules. ⁹

Rating and reputation system

Do not rely only on 5-star averages. In low-volume local marketplaces, star ratings are too noisy.

Use a **reputation score** with these inputs:

- completion rate,
- on-time arrival percentage,
- complaint severity,
- repeat-customer rate,
- admin quality flags,
- quote accuracy,
- cancellation/no-show rate.

Operationally, display a simple label to customers:

Label	Meaning
New	fewer than 5 completed jobs
Reliable	good completion and low complaint history
Top Local Pro	high repeat rate, strong punctuality, low dispute rate
Under Review	temporary soft-block pending investigation

This creates a better trust signal than raw stars alone.

Complaint and dispute system

KaamSetu should define **four complaint severities**:

Severity	Example	Response target
Critical safety	harassment, unsafe conduct, property threat	immediate
Live-job operational	no-show, wrong worker, price escalation during job	within 30–60 minutes
Quality issue	poor repair, unfinished task	same day
Billing issue	payment mismatch, token adjustment	within 24 hours

Each complaint should require a structured case file:

- job ID,
- booking type,
- worker assigned,
- OTP evidence,
- before/after photos,
- voice or written customer statement,
- admin disposition,
- refund/credit/warning outcome.

Under DPDP, KaamSetu must also maintain a grievance-redress mechanism, publish a point of contact, and be able to answer a data principal's questions about their data processing. These legal duties reinforce the need for a real complaints desk and not just an email inbox. ¹⁰

Payment architecture

This is the most important implementation decision in the document.

For v1, KaamSetu should **avoid collecting the full worker-service consideration in most cases**. The CBIC GST FAQ states that TCS applies where the consideration for taxable supplies is collected by the e-commerce operator, and the operator must then file GSTR-8 and remit TCS; the FAQ also states that suppliers making supplies through such operators can lose threshold simplicity where Section 52 applies. A later GST update also noted that service suppliers up to the threshold may be exempt from compulsory registration when

supplying through e-commerce platforms, and mandatory registration is required only for e-commerce operators required to collect TCS. Because the detailed structure matters, KaamSetu should have a GST counsel validate the final design before launch, but from an execution standpoint the safest bootstrap model is still: **KaamSetu collects its own booking/support fee; customers pay workers directly for most third-party jobs.** ¹¹

That leads to the correct v1 payment split:

Job type	KaamSetu collects	Worker collects	Why
fixed-price small task	booking/support fee; optional full payment only in selected categories later	usually service amount in v1	reduces GST/TCS and float complexity
quote-based job	inspection/site-visit token	project amount	quote scope changes are easier to handle
shift-based labour	booking token; maybe advance in B2B cases	remaining shift payment	lowers no-shows without full marketplace escrow

Use a **licensed payment aggregator** for all digital collections. Cashfree publicly states that it is an RBI-authorized payment aggregator, supports payment links, forms, QR, payouts, escrow products, and identity verification tools, and includes payment links/forms/softPOS with the gateway. That makes it a strong v1 option for KaamSetu. Cashfree's current public pricing is roughly **1.6% promotional / 1.95% standard** for many domestic modes, while Razorpay's public pricing is **2% + GST** with no setup or AMC. Either works, but Cashfree's split/escrow and KYC tooling fit the longer-term marketplace path slightly better. ¹²

Also: **do not build a wallet.** RBI's payment-system FAQ makes clear that operating a payment system requires authorization under the PSS Act, and payment-aggregator infrastructure is already regulated. KaamSetu should use licensed rails, not invent a stored-value product. ¹³

Cash handling strategy

Cash is not a bug in the KaamSetu model; it is a required launch mode.

The right v1 cash policy is:

- allow **cash at job completion**,
- require worker to mark cash collected in app,
- require customer confirmation or OTP completion,
- show **cash vs digital split** in worker scorecard,
- reconcile KaamSetu's own booking/support fee separately,
- do **weekly exception review** on jobs marked cash but later disputed.

This preserves Bharat-first usability without losing records.

Closed beta architecture

The beta should be gated on **both app distribution and backend access**.

Google Play supports internal testing for up to **100 testers**, and closed testing through email lists or Google Groups; Google also notes that new personal developer accounts must meet a 12-tester, 14-day closed-test requirement before production distribution. KaamSetu should therefore plan beta distribution carefully and preferably create the Play account under the business entity once the legal structure is ready, because organization accounts require organization registration documents plus a government-issued ID from an authorized representative. ¹⁴

Backend beta controls should include:

- invite code required,
- city + microzone allowlist,
- category allowlist,
- daily booking cap,
- worker cap per category,
- feature flags by cohort,
- panic switch to pause new jobs by zone.

System architecture blueprint

Tech stack recommendation

The best bootstrap stack for KaamSetu is:

Layer	Recommendation	Why
mobile apps	Flutter	one codebase, strong Android support, good localization
backend	Django + Django REST Framework monolith	fastest path to internal tooling + API + admin
database	PostgreSQL + PostGIS	relational integrity + geospatial dispatch
queue/jobs	start with cron + DB jobs, add Celery/Redis only after load warrants it	lower complexity
admin UI	Django Admin first, custom ops screens second	ship fast
infra	single AWS Lightsail instance in Mumbai region for phase 1, with daily backups	cheap and simple
edge/security	Cloudflare in front; move to Pro when public traffic grows	low-cost WAF/CDN

Layer	Recommendation	Why
object storage	S3-compatible bucket for docs/photos	keep sensitive files off app disk
messaging	WhatsApp Business app in phase 0; Cloud API in phase 1	faster validation, then automation
voice	simple IVR / virtual number	assisted booking and trust
payments	Cashfree or Razorpay	licensed, fast to launch

Flutter's official docs support standard internationalization workflows; Django's admin is specifically designed for internal management; PostgreSQL supports JSONB and text search; PostGIS adds geospatial indexing and query capability. Together, that stack is enough for a dense, one-city marketplace without premature microservices. ¹⁵

Why a monolith is the correct choice

KaamSetu should start with **one deployable monolith**. A microservices architecture is a bootstrap tax.

A monolith here gives five advantages:

- one codebase,
- one database,
- one permission model,
- one audit trail,
- one ops team.

That is exactly what a founder-led small engineering team needs.

Database blueprint

Use a relational schema first. The core tables should be:

Table	Purpose
users	parent identity table
customers	customer profile, language, addresses
workers	worker profile, trade, verification tier, bank status
worker_documents	KYC docs, expiry, review status
worker_skills	skill tags, level, categories
worker_zones	pincode/locality/service radius
worker_availability	online state, working hours, leave

Table	Purpose
invites	invite-only onboarding
jobs	master booking record
job_items	task lines, fixed-price items, shift lines
quotes	quote submissions for variable work
dispatch_attempts	match, notify, accept, reject, timeout
job_events	immutable job state log
otp_events	start/end OTP logs
payments	booking fee, tokens, payment-link status
cash_entries	cash-collected confirmations
ratings	customer and worker feedback
complaints	issue records and SLA clock
settlements_or_dues	only for KaamSetu fee reconciliation, not wallet balance
b2b_accounts	local business accounts
translations	category labels and templates by language
admin_actions	who changed what and when

The data model must also support **event logging**. Every status change should create a row in `job_events`. This will later save KaamSetu from endless “he said / she said” chaos.

Backend module blueprint

The backend should be split into modules, not services:

Module	Responsibility
auth	phone number, role, sessions, invite checks
worker onboarding	KYC, documents, references, approval
catalog	categories, fixed tasks, shift cards, service rules
booking engine	create/edit/cancel jobs
dispatch engine	worker matching and sequencing
quote engine	site visit, estimate submission, approval
payment module	booking fee, tokens, payment link status

Module	Responsibility
dispute module	complaints, evidence, outcomes
notifications	SMS, WhatsApp, push, IVR callbacks
analytics	fill rate, response rate, complaints, repeat jobs
admin audit	immutable state changes

What to automate first and what to keep manual

Automate immediately:

- OTP verification,
- invite-code gating,
- worker eligibility filtering,
- dispatch queue generation,
- notification sending,
- payment-link creation,
- job-event logging,
- SLA timers for open complaints.

Keep manual at first:

- final worker approval,
- pricebook changes,
- high-value quote approval,
- dispute resolution,
- refunds and waivers,
- B2B pricing,
- worker suspensions/restorations,
- new-category launch readiness.

This is the right balance between software leverage and operational control.

Security and compliance baseline

DPDP requires lawful processing, notice, specific informed consent, reasonable security safeguards, breach intimation to the Board and affected data principals in the event of a personal-data breach, and erasure when the purpose has ended or consent is withdrawn unless retention is legally required. For children, verifiable parental consent is required and tracking/targeted advertising restrictions apply. KaamSetu should therefore avoid child workflows entirely in the MVP, collect only necessary data, separate KYC documents from core app tables, encrypt sensitive files, and give users a clear delete/request flow from the start. ¹⁶

At the edge layer, Cloudflare's WAF is available across plans and Pro starts around **\$20/month when billed annually**, which is enough for public launch hardening after internal beta. ¹⁷

Launch engine

Worker acquisition system

The first 100 workers should not come from digital ads. They should come from **trusted offline nodes**:

- hardware and electrical shops,
- plumbing material stores,
- AC spare-parts shops,
- labour chowks,
- mason/welder contractor referrals,
- apartment maintenance supervisors,
- warehouse contractors,
- building-material dealers.

The worker acquisition field script should be simple:

1. explain what KaamSetu is,
2. explain it is invite-only and local,
3. collect trade, locality, phone,
4. explain verification tiers,
5. schedule onboarding camp,
6. do skill demo and ID capture,
7. activate only after review.

The message is not “earn unlimited income.” The message is: **“Get direct local work, less waiting, more repeat jobs, and less haggling.”**

First 100 workers plan

A realistic first-100-worker composition is:

Category	Target workers
electricians	20
plumbers	20
carpenters	15
appliance / AC technicians	15
helpers / labour / movers	20
painters / masons / welders for quote jobs	10

Do not try to make all 100 active at once. Aim for **60 verified, 40 activated, 25 high-response, and 15 “known reliable” core workers.**

First 100 jobs plan

The first 100 jobs should not be left to public demand generation. They should be manufactured through founder-controlled channels.

Recommended split:

Source	Target jobs
founder and team network households	30
friends-of-friends and referral households	20
apartment / colony groups	15
local shops / clinics / schools / offices	15
warehouses / builders / small contractors	10
WhatsApp inbound / walk-in demand	10

The point of these first 100 jobs is not growth. It is **learning**:

- actual arrival times,
- real price objections,
- repeat behaviour,
- complaint patterns,
- category-level unit economics,
- worker no-show patterns.

Supply-demand balancing strategy

The cardinal rule is: **launch by microzone, not by city limits.**

Each microzone should open only when it has:

- at least **5 active workers** in the top household categories,
- at least **2 reliable workers** per high-frequency category,
- one local ops fallback,
- known fixed-price ranges.

Operational targets for v1:

Metric	Healthy beta target
acceptance rate on top-3 dispatch	50%+
same-day fill rate	80%+ in active zones
no-show rate	under 8%

Metric	Healthy beta target
dispute rate	under 5%
repeat customer rate by month 3	25%+
jobs per active worker per week	3–6

If fill rate drops below target, **pause customer acquisition before adding more demand**. Ops discipline matters more than growth.

City launch playbook

The right first city is **not** the “best market.” It is the city where the founder can assemble trust the fastest.

City selection rule:

1. founder has personal network,
2. at least one field executor is local,
3. workers can be onboarded through referrals,
4. one local support number is recognized,
5. repeat B2B demand exists nearby.

The launch footprint should be **3 microzones, 5–6 categories, and one support number**. Do not launch the whole city. Do not launch all categories. Do not run citywide ads.

What must be tested before public launch

Before public release, KaamSetu must test:

- OTP start/end failure cases,
- double-booking prevention,
- worker offline/online sync,
- cash-entry fraud edge cases,
- customer cancellation after dispatch,
- quote revision workflow,
- complaint SLA timers,
- WhatsApp fallback when app install fails,
- IVR routing,
- payment-link failures,
- worker suspension mid-job,
- zone blocking,
- invite-code misuse,
- photo upload failures on slow connections,
- data deletion/export requests,
- breach-response escalation path.

Only after these flows are stable should the company move from invite-only closed beta to public waitlist.

Economics and build plan

Cost control plan

KaamSetu should consciously separate **required spend** from **ego spend**.

Do not spend on:

- office lease,
- heavy branding agency work,
- iOS in phase 1,
- broad digital ads,
- multiple gateway integrations,
- custom call-center integrations beyond what one IVR number needs,
- advanced BI stack.

Spend only on:

- one reliable server,
- the apps,
- the admin panel,
- communications stack,
- one support number,
- one payment gateway,
- field acquisition and verification,
- dispute-handling capacity.

Monthly operating cost estimate

A realistic **lean monthly run-rate excluding founder salary** is:

Cost head	Lean monthly estimate
full-stack / backend engineer	₹70,000–₹90,000
Flutter engineer or contractor	₹50,000–₹80,000
ops + support executive	₹20,000–₹30,000
field onboarding / acquisition	₹20,000–₹30,000
cloud, storage, monitoring	₹8,000–₹20,000
WhatsApp, OTP, IVR, voice	₹4,000–₹12,000
travel, legal, accounting, incident buffer	₹15,000–₹30,000
refunds / credits / contingencies	₹10,000–₹20,000

That produces a planning range of roughly **₹2.0 lakh to ₹3.1 lakh per month**, with a strong “founder-lean” operating target around **₹2.3–₹2.4 lakh per month**.

The infrastructure component is modest by design. AWS Lightsail publicly lists low-cost bundles, including a **\$20/month** 4 GB plan and smaller options below that, while Google Play charges a **\$25 one-time registration fee** and Apple charges **\$99 per membership year**. This is why Android-only is the correct v1 choice. ¹⁸

Communication tooling is also manageable. Public vendor pricing shows that:

- a basic cloud-contact-center / IVR setup can start around **₹1,999–₹3,499 per month**,
- WhatsApp BSP plans can be about **₹500 per number per month** plus per-template charges,
- India utility/authentication WhatsApp templates are about **₹0.115** each in one public rate card, and marketing templates about **₹0.8631**,
- SMS pricing can be around **₹0.18–₹0.25** per SMS in common entry tiers,
- and OTP voice can be around **₹1** per OTP. ¹⁹

Minimum revenue required to become self-sustaining

If KaamSetu’s lean monthly operating cost is approximately **₹2.35 lakh**, then the business should plan for at least **₹2.4 lakh monthly gross revenue** before calling itself self-sustaining.

A practical break-even model for v1 is:

Revenue stream	Example monthly contribution
1,500 completed jobs at ₹120 blended net revenue/job	₹1,80,000
120 paying worker memberships at ₹199	₹23,880
8 local B2B accounts at ₹3,500	₹28,000
Total	₹2,31,880

That is close to break-even but still tight. The implication is important: **consumer jobs alone are unlikely to cover costs early**. KaamSetu must build B2B recurring accounts and worker memberships alongside household demand.

The correct implementation lesson is therefore this: **self-sustaining does not mean “wait for marketplace magic.”** It means deliberately building a blended revenue base.

Payment-gateway cost decision

Between the two mainstream options reviewed, KaamSetu should default to **Cashfree first** unless merchant approval or support quality pushes the team elsewhere. Cashfree’s public materials show payment-link support, QR, splits, escrow products, payouts, and ID verification under one ecosystem, which is more aligned with where KaamSetu may go in phase 2 or 3. Razorpay is also perfectly viable, but its public price is currently simpler and higher at **2% + GST**. ¹²

Development phases

Phase 0

Manual validation before code hardens

Duration: 1–2 weeks.

Build almost nothing beyond:

- WhatsApp Business account,
- one IVR/virtual number,
- onboarding form,
- worker verification checklist,
- Google Sheet / admin stub,
- manual dispatch script.

Objective: validate categories, response times, price ranges, and complaint types.

Phase 1

Closed beta managed marketplace

Duration: weeks 3–8.

Build:

- admin panel,
- worker app,
- thin customer app,
- invite system,
- booking engine,
- dispatch engine,
- OTP start/end,
- payment recording,
- rating and complaint system.

No wallets. No masking. No fancy maps. No iOS.

Phase 2

Automation and controlled public opening

Duration: months 3–6.

Add:

- WhatsApp Cloud API flows,
- B2B account portal light,
- quote comparison,
- basic rebooking,
- worker membership paywall,
- payment-link automation,
- stronger analytics.

Phase 3

Selective scaling and trust hardening

Duration: after product reliability is proven.

Add:

- masked calling,
- selective platform-collected fixed-price jobs,
- escrow/split flows for specific use cases,
- stronger worker learning tools,
- selective new city replication,
- iOS only if real demand proves it.

The 30-day build plan

By day 30, KaamSetu should have:

- final category shortlist,
- verification SOP,
- invite-only worker pipeline,
- admin panel live,
- basic worker app test build,
- internal booking flow,
- manual dispatch process,
- one payment gateway account in sandbox,
- one WhatsApp business number,
- one IVR number,
- 20–30 verified workers,
- first 20–30 test jobs completed.

The 60-day build plan

By day 60, KaamSetu should have:

- Android closed-beta customer app,
- Android worker app stable enough for live jobs,

- microzone dispatch,
- complaint desk with SLA,
- OTP-protected jobs,
- payment links live,
- quote flow live,
- 50–70 verified workers,
- 60–80 completed jobs,
- early B2B account outreach.

The 90-day build plan

By day 90, KaamSetu should have:

- 100 invited workers in database,
- 40–60 active,
- 100+ completed jobs,
- 3 operating microzones,
- category-level pricebook v1,
- worker reliability baselines,
- no-show suppression tactics,
- one city dashboard,
- public waitlist readiness decision.

Risks and founder recommendation

Anti-copy strategy

KaamSetu's moat is **not the app UI**. The moat is:

- verified local worker graph,
- microzone dispatch know-how,
- discipline around who gets activated,
- multilingual operational scripts,
- real trust/reputation ledger,
- repeat B2B relationships,
- category-specific pricing memory,
- complaint-resolution muscle,
- local referral loops.

A better-funded clone can copy screens. It cannot easily copy **local supply density plus trust data plus operating habits**.

Top risks and failure modes

Risk	Early signal	Countermeasure
fake workers	KYC mismatch, unreachable references	invite-only onboarding, tiered verification
worker no-shows	multiple late starts	reliability score, temporary suspensions
bad first-job quality	early complaint spike	tighter category shortlist, skill test
demand too broad	low fill rate	microzone launch only
quote chaos	price disputes	quote templates and approval logs
worker bypassing platform	repeat off-platform contact	loyalty incentives, B2B lock-in, later masking
customer distrust	low conversion on first booking	assisted booking, local branding, verified labels
cash leakage	jobs marked cash without proof	OTP completion + customer confirmation
legal/GST confusion	CA flags on invoicing	keep v1 collection model simple, get counsel sign-off
data leakage	docs exposed or overshared	separate KYC store, minimal data capture
early negative word-of-mouth	disputes in first 50 jobs	founder-level support on all early jobs
overbuilding	slipping delivery dates	freeze scope to must-haves
low worker retention	inactive after onboarding	guaranteed early jobs, fast support
thin revenue	jobs high, money low	add B2B and memberships early
ops overload	founder handling every issue	admin tooling first
category complexity	many edge cases	postpone painters/masons/welders as projects
release friction	testing delays	closed track planning from day one
underpriced jobs	worker refusal	live pricebook review every week
safety incident	high-severity complaint	panic protocol, immediate suspension rule
multi-city temptation	founder attention splits	no second city before first city is stable

Final founder recommendation

The right way to build KaamSetu is to **treat software as an enabler of disciplined local operations**, not as the product by itself.

The correct founder move is:

- launch **one city only**,
- start with **5–6 categories**,
- build **admin first, worker second, customer third**,
- keep **payments simple**,
- use **WhatsApp and IVR from day one**,
- avoid **wallets, full commissions, and open bidding at launch**,
- get to **100 verified workers and 100 completed jobs** before thinking about public expansion,
- and build **B2B recurring demand** much earlier than most consumer marketplace founders expect.

If KaamSetu follows this path, it has a credible shot at becoming a **small, dense, trusted, cash-disciplined local operating system**. If it instead tries to launch as a broad consumer app with too many categories, too much automation, too much public supply, and vague pricing logic, it will likely become an expensive support desk masquerading as a marketplace.

Open questions and limitations

A few items still require founder-side or counsel-side confirmation before engineering locks them in:

- **GST structure**: the broad execution recommendation is to avoid collecting the main service consideration in v1, but the exact invoicing/TCS design should be signed off by a GST professional. ¹¹
- **Worker KYC mix**: the safest general recommendation is offline Aadhaar XML/QR or alternative IDs, but the exact acceptable-document policy should be written with legal review. ⁸
- **Voice and masking vendor choice**: IVR pricing is public, but call masking often requires a custom commercial plan. ²⁰
- **Launch-city category rates**: the final pricebook must be calibrated through actual local pilot jobs, not desk assumptions.
- **Entity structure and app-publishing setup**: Play Console account type and legal entity timing should be finalized before public release to avoid migration friction. ²¹

¹ ¹¹ <https://cbic-gst.gov.in/hindi/sectoral-faq.html>

<https://cbic-gst.gov.in/hindi/sectoral-faq.html>

² <https://developers.facebook.com/docs/whatsapp/guides/interactive-messages/>

<https://developers.facebook.com/docs/whatsapp/guides/interactive-messages/>

³ ¹⁰ ¹⁶ <https://www.indiacode.nic.in/bitstream/123456789/22037/1/a2023-22.pdf>

<https://www.indiacode.nic.in/bitstream/123456789/22037/1/a2023-22.pdf>

⁴ <https://developers.facebook.com/documentation/business-messaging/whatsapp/about-the-platform>

<https://developers.facebook.com/documentation/business-messaging/whatsapp/about-the-platform>

5 15 <https://docs.flutter.dev/ui/internationalization>

<https://docs.flutter.dev/ui/internationalization>

6 The Django admin site

https://docs.djangoproject.com/en/6.0/ref/contrib/admin/?utm_source=chatgpt.com

7 <https://postgis.net/>

<https://postgis.net/>

8 About Aadhaar Paperless Offline e-kyc - Unique Identification Authority of India | Government of India

<https://www.uidai.gov.in/en/ecosystem/authentication-devices-documents/about-aadhaar-paperless-offline-e-kyc.html>

9 <https://www.indiacode.nic.in/bitstream/123456789/16823/1/aA2020-36.pdf>

<https://www.indiacode.nic.in/bitstream/123456789/16823/1/aA2020-36.pdf>

12 <https://www.cashfree.com/payment-gateway-charges/>

<https://www.cashfree.com/payment-gateway-charges/>

13 <https://www.rbi.org.in/commonman/english/scripts/FAQs.aspx?Id=420>

<https://www.rbi.org.in/commonman/english/scripts/FAQs.aspx?Id=420>

14 <https://support.google.com/googleplay/android-developer/answer/9859348?hl=en>

<https://support.google.com/googleplay/android-developer/answer/9859348?hl=en>

17 <https://www.cloudflare.com/en-au/application-services/products/waf/>

<https://www.cloudflare.com/en-au/application-services/products/waf/>

18 <https://aws.amazon.com/lightsail/pricing/>

<https://aws.amazon.com/lightsail/pricing/>

19 20 <https://www.knowlarity.com/pricing/voice>

<https://www.knowlarity.com/pricing/voice>

21 <https://support.google.com/googleplay/android-developer/answer/13628312?hl=en>

<https://support.google.com/googleplay/android-developer/answer/13628312?hl=en>